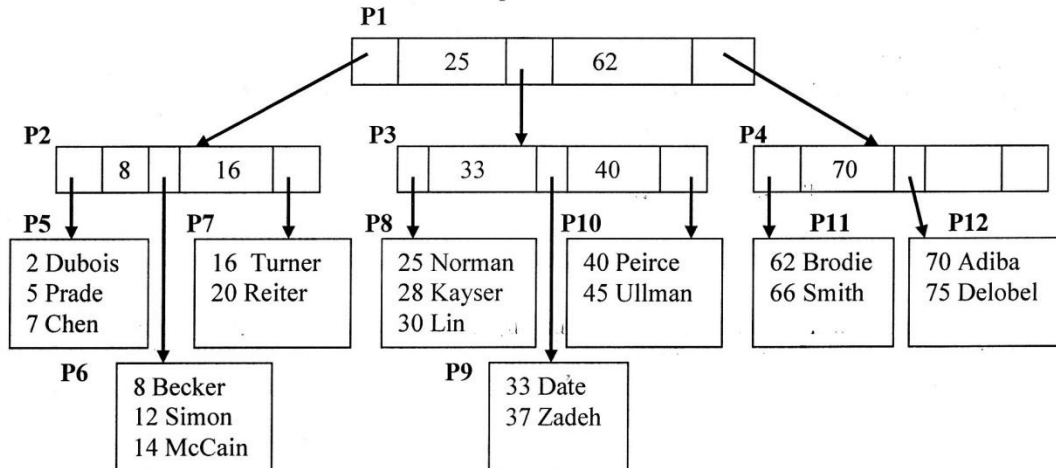
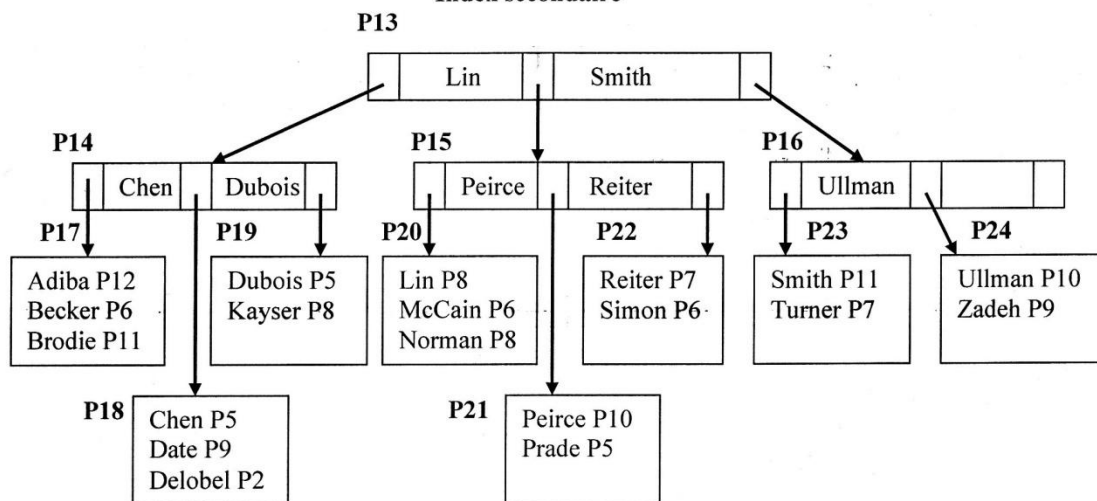


Soit la relation Auteur d'une BD bibliothèque sur laquelle est défini un index primaire sur le numéro d'auteur et un index secondaire sur le nom de l'auteur en utilisant la structure de B-arbre. Chaque B-arbre est d'ordre 3 et ses feuilles peuvent contenir au maximum 3 enregistrements. Nous avons obtenu les B-arbres suivants : (nous n'avons représenté du fichier que les propriétés numéro et nom d'auteur)

Index primaire



Index secondaire

**Questions :**

1. Représenter l'état de la relation et de ses index après insertion de l'auteur (27 Miranda).
2. Représenter l'état de la relation et de ses index après suppression de (20 Reiter) à partir de l'état initial.

B-ARBRE :

Un B-arbre d'ordre d (nombre de descendants directs d'un nœud interne) et de profondeur p est défini comme une arborescence ayant les propriétés suivantes :

- ☐ Chaque nœud a au plus d fils, c'est-à-dire d pointeurs.
- ☐ Chaque nœud excepté la racine et les feuilles a au moins $\lceil d/2 \rceil$ fils (plus petit entier non inférieur à $d/2$, ou encore l'arrondi supérieur de $d/2$).
- ☐ La racine a au moins 2 fils.
- ☐ Toutes les feuilles apparaissent au même niveau : (p) ,
- ☐ Un nœud ayant k fils ($k \leq d$) c'est-à-dire k pointeurs, contient $k-1$ clés.
- ☐ Les données (tuples) sont rangées dans les feuilles ou nœuds terminaux. Les nœuds non terminaux ne contiennent que des clés et des pointeurs vers d'autres nœuds de l'arborescence.

Les types d'accès fournis par un B-index

Index Principal : l'ordre physique coïncide avec l'ordre logique : il est construit au chargement initial de la relation et est défini sur une clé.

Index Secondaire : ordre physique est différent de l'ordre logique dans la plupart des cas et peut être défini sur des attributs non clés

Algorithme de recherche

Soit la recherche d'un tuple de clé c

- ☐ Lire la racine et rechercher quelle valeur de clé recouvre la valeur c .
- ☐ Lire la valeur de pointeur associé.
- ☐ Aller à la page pointée par le pointeur sélectionné.
- ☐ Rechercher quelle valeur de clé recouvre c , lire la valeur de pointeur associé et aller à la page pointée.
- ☐ Répéter l'opération précédente jusqu'à trouver la page feuille contenant la clé.

Algorithme d'insertion

Soit à insérer un tuple de clé c

- ☐ Rechercher la page feuille du B-arbre qui doit contenir ce tuple en appliquant l'algorithme de recherche : soit P_i cette page ; deux possibilités.
- ☐ Si P_i est non saturée alors insertion dans l'ordre des clés.
- ☐ Sinon il faut allouer une nouvelle page P' . On répartit les tuples de P_i avec le nouveau tuple en deux groupes équilibrés qui seront stockés respectivement sur P_i et P' .
- ☐ Soit P_0 la page qui pointait sur P_i ; deux possibilités :
- ☐ Soit P_0 est non saturée, c'est-à-dire qu'elle n'a pas les $d-1$ clés alors insertion d'un pointeur pour P' avec la valeur minimum de clé de P' .
- ☐ Sinon, il faut allouer une nouvelle page pour l'index tout en contrôlant le pointage du niveau antécédent. Dans le cas où plusieurs ancêtres de P_0 sont pleins alors l'insertion aura pour effet de modifier l'arbre sur plusieurs niveaux et ainsi être dans la possibilité de modifier la racine qui deviendra un nœud intermédiaire, et qui sera remplacée par une nouvelle racine avec 2 fils. La profondeur de l'arbre sera alors modifiée. On parle alors d'insertion avec éclatement de nœuds et propagation de l'éclatement jusqu'à la racine.

Algorithme de suppression

Soit à supprimer le tuple de clé c

- ☐ Appliquer l'algorithme de recherche de la clé c : soit P_i la page qui contient c
- ☐ Si P_i a un nombre d'enregistrement $\geq \lfloor d/2 \rfloor$ après suppression alors réaliser la suppression. Il faut cependant vérifier que la clé du tuple supprimé ne se retrouve pas comme clé dans l'index du B-arbre (vérifier tous les niveaux de l'index). Dans ce cas, il faut alors remonter dans la hiérarchie afin de remplacer cette clé par la clé du tuple suivant.
- ☐ Sinon (c'est-à-dire que P_i a moins de $\lfloor d/2 \rfloor$ fils après suppression) :
- ☐ On examine la page P_j immédiatement à gauche ou à droite de P_i et ayant le même père.
- ☐ Si P_j a plus de $\lfloor d/2 \rfloor$ enregistrements, on redistribue les enregistrements de P_i et P_j de manière équilibrée tout en conservant l'ordre. On répercute la modification sur les ancêtres de P_i puisque les clés sont modifiées.

Sinon on réalise une suppression avec fusion. On regroupe P_i et P_j en un seul bloc et on modifie les ancêtres de P_i . Cette fusion peut être récursive